Lab 2 Report: CUDA Laboratory

William Lebrato, aile20

1 Implementation

To ensure robustness across different hardware configurations and problem sizes, all three tasks utilize a **cyclical grid-stride indexing strategy**. Instead of assuming a one-to-one mapping between threads and data elements, threads iterate through the data using a stride equal to the total number of threads in the grid:

$$\texttt{stride} = \texttt{blockDim.x} \times \texttt{gridDim.x}$$

$$\texttt{index} = \texttt{threadIdx.x} + \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + k \cdot \texttt{stride}$$

This allows a specific kernel configuration (e.g., fixed block count) to process arrays of arbitrary size by having each thread process multiple elements sequentially if necessary.

1.1 Task 1: Single-Kernel Odd–Even Sort

The first version performs the full odd–even sorting algorithm inside a single CUDA kernel. Both the odd and even phases run back-to-back within that kernel, and since each phase depends on the previous one, the threads synchronize using `__syncthreads()` before continuing.

The work is divided by assigning each thread a set of element pairs using the cyclical grid–stride indexing described earlier. In every iteration, each thread compares its assigned pair and swaps the values if needed. A block-level barrier marks the end of each phase so that all comparisons complete before switching between odd and even steps.

Because `__syncthreads()` only synchronizes threads inside a single block, the entire algorithm must run within one block. This keeps the design simple: all phases occur within one continuous kernel call, with one shared view of the data and predictable synchronization.

1.2 Task 2: Multi-Kernel Odd–Even Sort

The second version removes the single-block restriction by moving phase control to the host. Instead of running all phases inside one kernel, the host launches one kernel for the odd phase and another for the even phase. Each kernel launch serves as a natural global synchronization point, allowing multiple blocks to be used safely.

Inside each kernel, the design matches that of Task 1: threads use cyclical grid–stride indexing to handle their portion of the element pairs. Since each kernel only performs one phase before returning, the host is responsible for launching the next phase. This guarantees that all threads across the entire grid finish one phase before the next begins.

The structure remains straightforward. Each kernel focuses solely on comparing and swapping its assigned pairs, while the host coordinates the progression between odd and even phases.

1.3 Task 3: Parallel Gauss–Jordan Row Reduction

To handle the full matrix size $N = 2048$, the static 2D arrays from the sequential version were replaced with flattened 1D arrays allocated dynamically on the host using `malloc`. All access to matrix elements is performed with row-major indexing ($i \cdot N + j$), matching the layout expected by the CUDA kernels.

The computation is divided into two CUDA kernels, which are launched once per pivot value k from 0 to $N - 1$:

1. **Normalize Kernel (`normalize_kernel`):** This kernel performs the normalization of the pivot row. Threads iterate over the columns $j > k$ in a grid–stride loop, dividing each element by the pivot coefficient. To avoid race conditions, a single thread updates the right-hand side value $y[k]$ and sets the pivot element $A[k][k]$ to 1.0.

2. **Elimination Kernel (`elimination_kernel`):** This kernel updates every row that is affected by the current pivot step. Each thread receives a row index through grid–stride iteration and determines its role based on the position of that row relative to the pivot. For rows below the pivot ($i > k$), the thread performs the elimination step, and for rows above the pivot ($i < k$), it performs the corresponding solving update. As the algorithm progresses, each pivot reduces the number of rows that require elimination while increasing the number of rows that need solving. The kernel handles this naturally by checking the row index.

All data is copied to the GPU before the main elimination loop begins and is transferred back to the host only after all pivot steps have finished.

2 Results

All sequencial implementations where ran with -O2 level of optimization

2.1 Odd–Even Sort (Tasks 1 and 2)

The input size for the sorting implementations was 2^{19} elements.

Table 1: Execution Times for Odd–Even Sort

Configuration	**Time (s)**	**Speedup**
Sequential	551.47	1.00
Task 1	52.58	10.49
Task 2	23.08	23.89

2.2 Gauss-Jordan Row Reduction (Task 3)

The sequential Gauss-Jordan row reduction was executed 5 times on a 2048×2048 matrix.

Table 2: Execution Times for Gauss-Jordan Row Reduction (2048×2048)

Configuration	**Time (s)**	**Speedup**
Sequential	20.52	1.00
Parallel	3.65	5.62

3 Analysis

3.1 Task 1

For the single-kernel implementation, the work must be contained within one block because the algorithm relies on `__syncthreads()` for synchronization between phases. Since this barrier only works within a block, the kernel is launched with the maximum practical number of threads (1024) to cover as many element pairs as possible. This configuration balances simplicity and performance, using all available threads within the single-block constraint.

3.2 Task 2

In the multi-kernel implementation, phase synchronization is handled by the host through separate kernel launches. This removes the single-block limitation and makes it possible to use several blocks in parallel. Although a larger grid could have increased the degree of parallelism even further, the chosen configuration already provided a substantial speedup—roughly twice as fast as Task 1—so scaling up the block count was unnecessary for achieving strong performance in this case.

3.3 Task 3

For the Gauss–Jordan row reduction, all threads are utilized during every pivot step. The grid and block configuration used in the implementation provides enough threads to cover all 2048 rows through grid–stride iteration. As the pivot increases, fewer rows require elimination while more require solving, but the unified elimination kernel naturally adapts to this shifting workload. Each thread determines whether its assigned row is above or below the pivot and performs the appropriate update, which keeps the workload evenly balanced across the GPU throughout the computation.